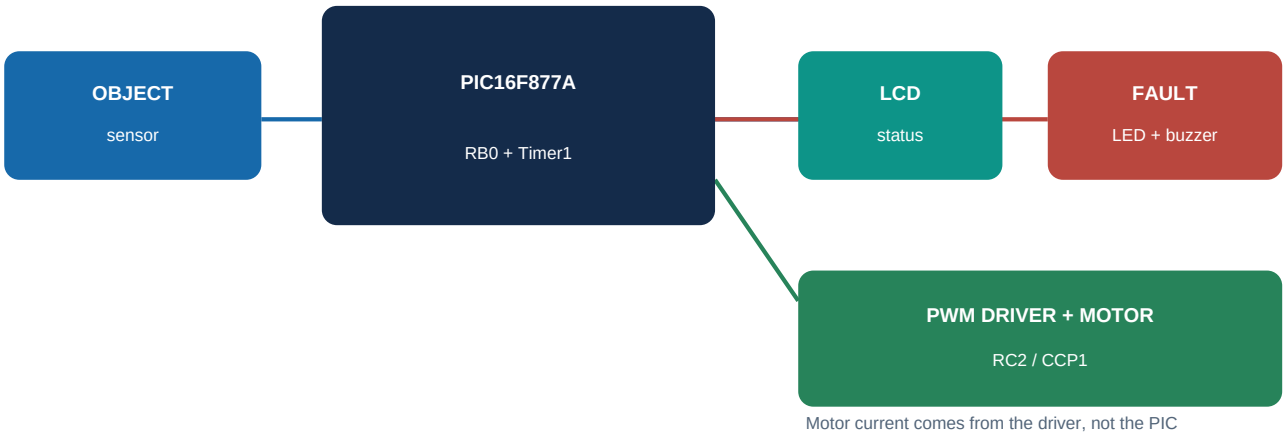
PIC16F877A

Conveyor Belt Monitoring & Control

A beginner-friendly guide to object sensing, PWM motor control, Timer1, and jam shutdown.



Motor current comes from the driver, not the PIC

WHAT YOU WILL LEARN	HOW THE EXAMPLE BEHAVES
Digital inputs, port direction, LCD, PWM, timer interrupts, and a simple jam response.	Count an arriving object. If it stays at the checkpoint too long, stop PWM and latch a fault.
CLOCK	PROJECT FILE
20 MHz crystal / _XTAL_FREQ	newmain.c | PIC16F877A | MPLAB X / XC8

Project scope
This is a classroom example, not a certified safety controller. Match the Proteus or hardware clock to 20 MHz.

01 Project overview

The program supervises one conveyor checkpoint using a PIC16F877A.

A digital IR/proximity module tells the PIC whether an object is at the sensor. CCP1 creates PWM for a motor driver. Timer1 measures how long the sensor stays blocked. If the object lingers, the firmware stops PWM, shows a jam, and turns on fault outputs.

INPUT / OUTPUT	JOB
RB0 sensor	Detect object present (active LOW by default).
Timer1 interrupt	Build an approximate millisecond counter.
RC2 / CCP1 PWM	Set motor-driver enable duty; default is 60%.
LCD, RB1, RB2	Show state and indicate a jam.
RB3 button	Stop belt; acknowledge jam only after sensor clears.

Main-loop sequence

STEP	PROGRAM ACTION
1	Read the sensor and detect a clear-to-blocked transition.
2	Increment object count once and remember start time.
3	Compare elapsed time with JAM_TIME_MS (5,000 ms by default).
4	Latch a jam; otherwise run unless STOP is held.
5	Update fault outputs and refresh the LCD about every 200 ms.

Safety
The RB3 push-button is not an emergency stop. Real conveyors require guards and independent safety-rated shutdown controls.

02 Wiring and PIC pins

Connect as shown by the source. Confirm sensor polarity before operation.

PIC PIN	SIGNAL	CONNECTION / NOTE
RB0	Object sensor	Digital output; active LOW by default.
RB1	Fault LED	Use a series current-limiting resistor.
RB2	Buzzer	Use a transistor or driver module.
RB3	Stop / acknowledge	Button to GND; external pull-up required.
RC2/CCP1	PWM	Connect to motor-driver enable/PWM input.
RD2, RD3	IN1, IN2	Direction inputs; source selects one direction.
RD0, RD1	LCD RS, E	LCD control.
RD4-RD7	LCD D4-D7	4-bit data bus; LCD R/W tied to GND.
VSS / VDD	PIC supply	Use correct regulated supply; share logic ground.

Signal path

SENSOR OUT	RB0
RC2 PWM	Driver EN/PWM
RD2 + RD3	Driver IN1 + IN2
Driver OUT	Low-voltage DC motor
LCD	RS=RD0, E=RD1, D4-D7=RD4-RD7, R/W=GND

Motor connection
Never connect a motor directly to a PIC pin. Use a driver rated for motor voltage and current, suitable protection, and a common signal ground.

Clock and reset wiring

OSC1 / OSC2	Connect the 20 MHz crystal across the PIC oscillator pins; choose loading capacitors from the crystal and device requirements.
MCLR / VPP	Keep reset inactive with a pull-up to the PIC supply, following the PIC data sheet.
Power-up check	Confirm VDD/VSS pins, local decoupling, and common logic ground before enabling the motor driver.

03 Ports and registers

A register is a control or data location inside the PIC.

REGISTER	PURPOSE	PLAIN-LANGUAGE VIEW
TRISB	RB0/RB3 inputs; RB1/RB2 outputs.	Pin direction: 1=input, 0=output.
PORTB	Read inputs and set output levels.	The pin HIGH/LOW values.
TRISC2	Set RC2 as PWM output.	Direction of the CCP1 pin.
TRISD / PORTD	LCD and motor direction signals.	Low bits control, high bits carry LCD data.
ADCON1	Select digital I/O behavior.	Turns analog-capable pins into digital pins here.
CCP1CON	Select PWM mode and low duty bits.	Configure CCP to generate pulses.
PR2 / T2CON	Set PWM period; start Timer2.	Choose pulse repetition rate.
CCPR1L	Upper PWM duty bits.	Set fraction of the period ON.
T1CON	Timer1 clock and prescaler.	Choose timer clock, divider, and run state.
TMR1H / L	Timer1 count and reload.	The 16-bit timer value.
PIE1 / PIR1	Enable and detect Timer1 event.	Interrupt enable and event flag.
INTCON	Enable interrupt response.	Master interrupt switches.

TRISB example
TRISB = 0x09 = 0000 1001 binary. Bits RB0 and RB3 are 1, so they are inputs; other PORTB bits are outputs.

Read the bit pattern

BIT	RB7 ... RB4	RB3	RB2	RB1	RB0
TRISB	0000	1	0	0	1
Meaning	not used here	button input	buzzer output	LED output	sensor input

Peripheral setup examples

SETTING	EFFECT
ADCON1 = 0x06	Use the shared pins as digital I/O for this design.
T1CON = 0x30	Timer1 is configured with its internal clock and 1:8 prescaler while initially stopped.
T1CONbits.TMR1ON = 1	Start Timer1 after loading the counter.
PIE1bits.TMR1IE = 1	Allow Timer1 overflow to request an interrupt.
INTCONbits.PEIE / GIE	Enable peripheral and global interrupt handling.

04 Timer1 and PWM calculations

The timing follows from the 20 MHz crystal and the selected prescalers.

Timer1: approximate 1 ms tick

CALCULATION	RESULT
Oscillator Fosc	20,000,000 cycles/s
Instruction clock Fosc / 4	5,000,000 ticks/s
Timer1 prescaler	1:8; one count = 8/5,000,000 = 1.6 us
Counts per 1 ms	1,000 / 1.6 = 625 counts
Preload	65,536 - 625 = 64,911 = 0xFD8F

Timer1 overflows, the interrupt reloads 0xFD8F, then increments system_milliseconds. Five thousand ticks represent about five seconds. ISR execution adds a small timing error.

CCP1 PWM: approximately 5 kHz

```
PWM frequency = Fosc / (4 x Timer2 prescaler x (PR2 + 1))
              = 20,000,000 / (4 x 4 x 250) = 5,000 Hz

PR2 = 249; Timer2 prescaler = 1:4
Default duty = MOTOR_SPEED_PERCENT = 60%
```

The duty helper converts a 0-100 percentage into a 10-bit CCP value. At 0%, no ON time is sent to the driver. At 60%, the PWM signal is ON for about 60% of each cycle. Actual belt speed depends on motor, driver, supply, and load.

Timing is approximate
This software timebase is suitable for the learning timeout. Precision industrial timing needs measurement and calibration.

What happens during one timer tick

1. Count	Timer1 advances at one count per 1.6 us.
2. Overflow	After 625 counts from the preload, Timer1 sets its interrupt flag.
3. Interrupt	The CPU briefly runs timer1_interrupt().
4. Reload	The handler loads 0xFD8F again and increments system_milliseconds.
5. Compare	The main loop subtracts object_start_time from current_time.

Example: if an object started at 12,400 ms and the current reading is 16,900 ms, elapsed time is 4,500 ms, so a 5,000 ms jam threshold has not yet been reached. At 17,400 ms, elapsed time is 5,000 ms and the jam is latched.

05 Program walkthrough

Follow main(), then open each helper function it calls.

FUNCTION / SECTION	WHAT IT DOES
Settings and pin names	Set 20 MHz, sensor polarity, 60% PWM, jam time, and readable pin aliases.
LCD functions	Initialize 4-bit mode and send commands/characters.
motor_set_speed	Convert requested percentage into CCP duty bits.
pwm_initialize	Set PR2, CCP1 PWM mode, and Timer2.
conveyor_set_running	Apply selected duty or set duty to zero.
timer1_interrupt	Reload Timer1 and add one to software time.
timer1_get_milliseconds	Safely copy the shared 16-bit time value.
update_object_and_jam_status	Count sensor edges and latch a long detection.
display_status / update_fault_outputs	Show state and control LED/buzzer.
main	Configure I/O, start peripherals, repeat the control loop.

Jam test, in plain language

```
if (sensor_active && !jam_latched)
{
    if ((current_time - object_start_time) >= JAM_TIME_MS)
    {
        jam_latched = 1;
    }
}
```

Elapsed time is current time minus object start time. The jam flag stays set after the sensor clears. It is acknowledged only if RB3 is pressed while the sensor is clear.

Motor power
RC2 is a logic signal. The external driver switches motor current; the PIC does not power the motor.

Important program variables

VARIABLE	PURPOSE
system_milliseconds	Volatile time counter updated by Timer1 interrupt.
object_start_time	Timestamp captured when the sensor first detects an object.
previous_sensor_active	Remembers the previous sensor state so one object is counted once.
object_count	Number of clear-to-active transitions; display rolls through 000-999.
jam_latched	Keeps a jam active until the sensor is clear and STOP is pressed.
motor_running	Records whether the program has applied nonzero PWM.

06 LCD states and safe test sequence

Test sensor and jam behavior before connecting a motor.

DISPLAY / OUTPUT	MEANING
BELT:RUN OBJ:nnn	Belt enabled; object count shown.
STOPPED OBJ:nnn	RB3 is held low; PWM is off.
BELT:JAM OBJ:nnn	Dwell threshold exceeded; fault is latched.
STOP:PRESS RESET	Clear sensor, then press RB3 to acknowledge.
RB1 + RB2	LED and buzzer outputs on during latched jam.

Suggested lab sequence

TEST	EXPECTED RESULT
Power on with sensor clear	Startup message, then BELT:RUN.
Block sensor briefly then clear	Object count goes up once; no jam.
Hold sensor active for over 5 s	PWM stops, JAM appears, LED/buzzer turn on.
Clear sensor and press RB3	Jam latch clears; releasing button restarts belt.
Hold RB3 during normal operation	PWM stops and STOPPED is displayed.

Before the motor: use simulation or an LED stand-in for PWM. Verify sensor logic polarity, clock setting, LCD wiring, driver logic voltage, common ground, motor current, and driver rating. RB3 is a normal input, not an emergency-stop circuit.

Jam recovery state

SENSOR	RB3 BUTTON	RESULT
Blocked	Any	Jam remains latched; PWM stays off.
Clear	Released	Jam remains latched until acknowledged.
Clear	Pressed (LOW)	Jam latch clears; button still holds the belt stopped.
Clear	Released again	Belt may run again.

07 Settings, common mistakes, glossary

Quick reference while studying or changing the example.

SETTING	DEFAULT	CHANGE WHEN
_XTAL_FREQ	20,000,000UL	Oscillator frequency changes.
MOTOR_SPEED_PERCENT	60	Choose another 0-100% duty.
JAM_TIME_MS	5,000	Change allowed object dwell time.
SENSOR_ACTIVE_LEVEL	0 (LOW)	Sensor output is HIGH during detection.
Timer1 preload	0xFD8F	Clock or prescaler changes.

Common mistakes

- Driving motor/buzzer directly from a PIC pin.
- Forgetting shared ground between PIC and driver logic.
- Wrong sensor active level or missing RB3 pull-up.
- Proteus clock differs from _XTAL_FREQ.
- Assuming 60% PWM means a fixed mechanical speed.
- Treating the button as a safety-rated emergency stop.

Glossary

TERM	MEANING
GPIO	General-purpose input/output pin.
PWM	Pulse-width modulation: ON/OFF pulses with adjustable duty.
Duty cycle	Percent of a PWM period spent ON.
Prescaler	Divider that slows a clock before a timer.
Interrupt	Hardware event that runs a short handler.
Latch	Stored state that remains until an explicit reset condition.
H-bridge	Driver circuit that switches motor current and direction.

Source and verification note

This handbook follows newmain.c and explains its intended operation. The firmware has not been compiled or hardware-tested in this handbook workflow. Build it with your MPLAB X / XC8 installation and verify it on a safe setup before use.

Educational use
Production machinery requires a risk assessment, proper guarding, a safety-rated emergency stop, and independent safety controls.